Rapport Technique : Implémentation de l'Algorithme ChaCha20

Équipe :

AKHDARI Ilyes – MOUHSINI Ilyes – ZERROUK Achraf – YOUSSOUFI Marouane – BOULAKHRAS Mohamed

Application web de gestion d’un parc informatique

1. *Introduction*

Dans le cadre de la sécurisation de notre plateforme web de gestion de parc informatique, la confidentialité des données d'authentification est un enjeu majeur. Conformément aux exigences de la SAÉ, ce projet vise à implémenter "from scratch" (sans dépendance librairie externe) un algorithme de chiffrement moderne.

Il est ici question de **ChaCha20**, un algorithme de chiffrement par flot (*stream cipher*) développé par Daniel J. Bernstein et standardisé par l'IETF via la **RFC 8439**. Réputé pour son excellente performance logicielle et sa résistance à la cryptanalyse, il constitue aujourd'hui un standard de l'industrie (utilisé notamment dans TLS 1.3). Ce rapport détaille le fonctionnement mathématique de l'algorithme, décrit notre implémentation en Python, ainsi que la validation croisée de nos résultats.

2. *Description Théorique de l'algorithme*

Contrairement aux chiffrements par blocs (comme l'AES) qui nécessitent un bourrage (*padding*) des données, ChaCha20 génère un flux continu et infini de données pseudo-aléatoires appelé **Keystream**. Ce flux est ensuite superposé au message clair.

2.1 La Matrice d'État (The State)

Le moteur de ChaCha20 repose sur une matrice de 16 mots de 32 bits (4x4), représentant un état total de 512 bits. Elle est structurée très précisément :

- **Constantes (Mots 0-3) :** La chaîne hexadécimale correspondant à *"expand 32-byte k"*. Elle fournit une asymétrie de départ.
- **Clé (Mots 4-11) :** La clé secrète de 256 bits partagée entre les utilisateurs.
- **Compteur (Mot 12) :** Entier de 32 bits incrémenté à chaque bloc de 64 octets généré, assurant que le keystream ne boucle jamais sur lui-même pour un même message.
- **Nonce (Mots 13-15) :** Un vecteur d'initialisation (IV) unique de 96 bits.

2.2 Le Mécanisme ARX et l'Effet Avalanche

Le cœur cryptographique repose sur la fonction **Quarter Round**, qui modifie 4 mots de la matrice en utilisant l'architecture **ARX** :

1. **Addition** (modulaire 32 bits) : Apporte la confusion mathématique.
2. **Rotation** (décalage circulaire gauche) : Assure la diffusion des bits.
3. **XOR** (Ou Exclusif) : Introduit la non-linéarité, empêchant la résolution par un système d'équations linéaires.

L'algorithme effectue **20 itérations** (Rounds) de ce mélange : 10 tours sur les colonnes, alternés avec 10 tours sur les diagonales. Cette répétition garantit un **Effet Avalanche** total : la modification d'un seul bit en entrée (clé ou nonce) modifie statistiquement 50% des bits de sortie. Après ces 20 tours de mélange garantissant l'effet avalanche, l'algorithme effectue

une étape cruciale : **il additionne l'état initial à l'état final**. Cette addition transforme la permutation réversible des 20 tours en une véritable fonction à sens unique, empêchant un attaquant de remonter le calcul à l'envers pour retrouver la clé.

3. *Architecture et Déroulement de l'Implémentation*

Pour répondre aux exigences de testabilité, nous avons adopté une architecture logicielle modulaire scindée en deux fichiers.

3.1 Le Module "Moteur" : `chacha20_fromscratch.py`

Ce fichier encapsule la classe orientée objet `ChaCha20`, qui implémente strictement la logique de la RFC 8439.

- **La méthode d'initialisation (`__init__`) :** Elle configure l'état initial. Elle convertit la clé (32 octets) et le nonce (12 octets) en mots de 32 bits exploitables par la matrice via la bibliothèque `struct`.
- **La gestion stricte des 32 bits (`_rotl`) :** Python gérant des entiers de taille arbitraire (infinie), une simple rotation de bits (`<<`) provoquerait un dépassement de mémoire (overflow). Nous avons implémenté la méthode `_rotl` avec un masque hexadécimal `& 0xffffffff`. Ce masque agit comme un troncateur strict, forçant le calcul à se comporter comme un registre CPU matériel de 32 bits.
- **La génération de blocs (`_generate_block`) :** C'est le chef d'orchestre de l'algorithme. Pour chaque bloc de 64 octets, cette méthode :
    1. Copie l'état initial dans un `working_state`.
    2. Applique les 20 rounds de mélange via `_quarter_round`.
    3. Additionne l'état final à l'état initial (pour empêcher la réversibilité de l'état interne).
    4. Sérialise la matrice finale en un flux d'octets. **Point crucial :** ChaCha20 impose le format *Little Endian* (octet de poids faible en premier). Nous utilisons `struct.pack('<16I', *working_state)` pour garantir la conformité binaire avec la norme.
- **Le traitement du message (`encrypt` / `decrypt`) :** La fonction découpe le texte clair en fragments (*chunks*), appelle le générateur de bloc pour créer le keystream correspondant, et applique l'opération XOR. Grâce à la propriété involutive du XOR (`(A ⊕ B) ⊕ B = A`), la méthode `decrypt` se contente de ré-appeler la méthode `encrypt`.

3.2 Le Script d'Intégration et Validation : `chacha20_library.py`

Une implémentation cryptographique "maison" n'a de valeur que si elle est prouvée exacte. Ce script agit comme un banc de test automatisé.

1. **Phase d'instanciation :** Définition d'une clé secrète, d'un nonce unique, et d'un mot de passe à chiffrer.

2. **Double exécution :** Le message est chiffré par notre classe `ChaCha20`, puis chiffré indépendamment par le module industriel `Crypto.Cipher.ChaCha20` (PyCryptodome) avec les mêmes paramètres.
3. **Validation croisée :** Une vérification stricte d'égalité est effectuée entre les deux flux de sortie. La concordance parfaite valide mathématiquement notre implémentation de la RFC.
4. **Simulation de persistance :** En prévision de l'insertion dans la base de données MariaDB (SAÉ Web), le flux d'octets chiffré est converti en chaîne hexadécimale via `.hex()`. Cela garantit que les caractères non-imprimables ne corrompent pas la base. Le résultat final est exporté dans le fichier de log `resultat_sae.txt`.

4. Conclusion

4.1 Retours d'expérience technique

L'implémentation bas niveau de ce chiffrement a mis en lumière plusieurs défis :

- **Le respect strict des RFC :** Une différence initiale de résultat avec la librairie de référence nous a montré qu'une “petite” erreur peut totalement fausser le chiffrement. Nous avions initialisé le compteur de bloc à `1`, alors que la norme impose un départ à `0`.
- **La manipulation binaire en Python :** Contrairement au langage C, le typage dynamique et la taille infinie des entiers en Python imposent une rigueur constante (utilisation de masques binaires, conversion d'endianness) pour simuler des opérations processeur.

4.2 Recommandations possibles (Mise en production)

- **Performance :** Notre code Python est pédagogique. Pour traiter de gros volumes de données sur un serveur, il faudrait utiliser une version pré-compilée en C (comme celle de PyCryptodome) ou exploitant les instructions vectorielles du processeur (SIMD).
- **Gestion des mots de passe :** Bien que l'exercice exige l'usage de ChaCha20, la norme de sécurité pour le stockage de mots de passe en base de données exige l'utilisation d'une fonction de hachage lente et asymétrique (comme **Argon2** ou **Bcrypt**). Un chiffrement symétrique comme ChaCha20 reste cependant parfaitement adapté au chiffrement des flux réseau de notre parc informatique ou à la protection de clés d'API réversibles.

5. *Bibliographie et Webographie*

- **Internet Engineering Task Force (IETF)** : RFC 8439 - *ChaCha20 and Poly1305 for IETF Protocols* (Spécifications techniques officielles).
- **Daniel J. Bernstein** : *ChaCha, a variant of Salsa20* (cr.yp.to) - Notes de conception de l'auteur.
- **Documentation Python**
- **Documentation PyCryptodome** : `Crypto.Cipher.ChaCha20` pour la méthode de validation.